Resend Take-Home: Customer Tickets - Ivan Dans

How I approached this: for each ticket I first wrote out my internal notes, separating what the customer actually told me from what I'm assuming and from what I still need to ask. I then wrote the customer response from those notes. At the end of each ticket I've added one idea for how the question could be prevented from coming back, because your handbook describes a ticket reaching a human as a signal that something further upstream should have handled it.

For the labels I used the categories in your handbook, which are Reliability, Usability, Functional, Deliverability, Abuse and Success Outreach, and then added the product area.

Priority order:

Priority	Ticket	Label	Why this position
1	RES-7921	Reliability / Sending	Thousands of login emails failed over four hours, which means real users couldn't get into their accounts. I'm treating it as a possible incident until the logs tell me otherwise. Escalated below.
2	RES-1348	Reliability / API authorization	Still failing right now, and the cause is unknown. It could be blocking all of their sending. I'd look at this in the same first sweep as the one above.
3	RES-3485	Usability / Rate limits	Notifications are failing right now, but the error message explains itself and there's an immediate fix.
4	RES-2196	Deliverability / Gmail	Real business impact, but the mail is arriving. I need evidence before I can say anything useful.
5	RES-1927	Usability / Domains	The customer is blocked partway through setup, but it's a quick and well-documented fix.
6	RES-5842	Usability / Receiving	A question about how to build something. Nothing is broken.
7	RES-2984	Usability / Sending	A very general question with no active failure. It's fast to answer, so it wouldn't actually sit for long.

A note on the ordering: this is about what I'd do first when I can't do everything at once, rather than which customer matters most. Tickets 5 to 7 only take a few minutes each, so if the queue allows I'd clear them quickly instead of letting them age behind the investigations.

Ticket RES-7921. "My emails suddenly stopped sending last night for 4 hours and thousands of magic links didn't send. What happened? This is unacceptable."

Label: Reliability / Sending. **Priority: 1.**

Note explaining my thinking:

What the ticket tells me: roughly four hours last night, a high volume of emails, and the emails are magic links, which means people were trying to log in and couldn't. The customer says it stopped, so the window is probably over.

What I don't know yet is the important part. There are three possibilities and each one belongs to a different team. The requests could have been rejected by our API, in which case their logs will show error codes. They could have been accepted by us and then failed somewhere after that, which would show up as emails with no delivery events against them. Or they could never have reached us at all, which would point to something on their side, such as a deploy or a broken queue. I don't want to guess between those three in front of a customer who is already upset.

What I'd do, in order:

1. Check the status page and the internal incident channel for last night before I reply at all. If we had an outage, I don't want the customer finding that out because I asked them questions about it.
2. Pull up the account's sending volume for that window. If it drops to zero and then comes back, with normal volume either side, that tells me the shape of the problem quickly.
3. Look at whether the emails were accepted or rejected. If our API returned IDs, I'd check the delivery events on a few of them. If it returned errors, I'd ask the customer for the status codes from their logs.
4. Check whether any other accounts show the same window. If they do, this isn't really a support ticket any more, it's an incident, and it needs a status page entry.
5. I wouldn't tell them to simply resend the failed batch. Magic links normally carry an expiry, though the ticket doesn't tell me what theirs is, so I'd want that confirmed before anybody resends anything. If those links have already lapsed, sending the batch again would deliver thousands of dead links to thousands of already frustrated users and create a second problem on top of the first. If a resend is needed, the links would have to be generated again first, and that's something they'd do in their own application.

One thing I want to be transparent about: I've put this above the ongoing 403 errors and it's a close call. This one is bigger and affects more people, but it's already over, whereas the 403s are still failing while I'm reading the ticket. I'd look at both in the same first fifteen minutes. I've put this one first because of the scale, because of the risk of it happening again tonight, and because understanding it might also explain the other one.

Draft response to the customer:

> Hi there,

> Thanks for reaching out, and I'm sorry this happened. My name is Ivan and I'll be looking into this with you. Four hours of failed magic links means your users weren't able to log in, so I understand why this is frustrating, and I'm treating it as a possible incident rather than a routine ticket.
>
> Here's what I'm doing on our side. I'm pulling your account's sending activity for last night in order to confirm the exact window, and I'm checking whether the emails were rejected by our API, accepted by us but not delivered, or never reached us at all. Those three possibilities have different causes, so I'd rather confirm which one it is than guess.
>
> There are two things that would help me get there faster, if you have them. The first is the approximate time window in UTC. The second is two or three email IDs or API responses from your logs during the failure.
>
> One thing I'd ask in the meantime: please don't resend last night's batch until we've confirmed how long your magic links stay valid. If they've already lapsed, resending would deliver thousands of links that no longer work, which would make things worse rather than better. If they have lapsed, or might lapse before they arrive, the links would need to be generated again first.
>
> I'll come back to you within the hour with whatever I've found, even if I don't have the full answer yet.
>
> Best, Ivan

How I would stop this coming back:

An alert on our side when an account's sending volume drops to zero compared to its own normal pattern. If a customer sends consistently and then stops for four hours, we should be the ones telling them, rather than hearing about it from them the following morning.

Ticket RES-1348. "I'm seeing a ton of 403 errors on my account. How do I fix that?"

Label: Reliability / API authorization. **Priority: 2.**

Note explaining my thinking:

The ticket tells me there are a lot of 403 errors and nothing else. A 403 means the request was refused, but the status code on its own doesn't tell me why, and I don't want to assume the API key is fine just because the request got that far. Resend returns a 403 for inactive and suspended keys as well as for scope and domain problems.

The documented causes each have a different fix. It could be an API key that's inactive or suspended, an OAuth access token missing the scopes the request needs, the testing restriction

that applies before a domain is verified, or sending from a domain that hasn't been verified yet. A direct HTTP request with no `User-Agent` header can also be refused before it reaches the API at all. None of those can be told apart from the number 403 on its own, which is why the first thing I need is the error body rather than more description.

One thing the status code does rule out. A key that's restricted to sending, used on an endpoint that isn't sending, comes back as a 401 rather than a 403. So the fact that they're seeing 403s tells me it isn't that case, which is worth knowing before I start asking them to check things.

While I wait for that, I'd check the domains on their account and their verification status from my side, since that answers about half of these without needing anything from the customer.

Draft response to the customer:

> Hi there,
>
> Thanks for reaching out. I'm Ivan, from the Resend support team, and I'm happy to help you get this sorted.
>
> A 403 means the request was refused, and there are a few different reasons that can happen, each with its own fix. Rather than have you try things one at a time, let's identify which one this is.
>
> Could you send me the full JSON error body from one of the failing requests? It looks something like this:
>
> `{ "statusCode": 403, "name": "...", "message": "..." }`
>
> The `name` and `message` fields together tell us which documented case we're dealing with. Please don't include your API key when you send it over, as I never need it in order to help you.
>
> While you're getting that, I'm checking your account from my side. The possible causes are an API key that's inactive or suspended, an OAuth access token missing the scopes it needs, sending to addresses other than your own before a domain is verified, or sending from a domain that hasn't been verified yet. Each of them has a different next step, so identifying which one it is saves you trying fixes at random.
>
> One more thing that would help: if these errors started suddenly after everything had been working, let me know roughly when. That timing can narrow things down considerably.
>
> Best, Ivan

How I would stop this coming back:

Every 403 response body could include a link to the documentation for that specific cause. The error already knows why it refused the request, so telling the customer at that moment would remove most of these tickets before they're written.

Ticket RES-3485. Rate limit error: "Too many requests. You can only make 2 requests per second."

Label: Usability / Rate limits. **Priority: 3.**

Note explaining my thinking:

The error message tells me this account's limit is 2 requests per second, so that's the number I'll work with. I wouldn't replace it with a remembered default, because limits vary by account and change over time, and the customer's own error message is better evidence than my memory.

The important thing here is that a request rejected with a 429 was never accepted, so no email was sent. That means retrying it can't produce a duplicate, and the fix is to slow down and retry rather than to change anything about the emails themselves.

It's also worth knowing that the limit applies across the whole team, so a second service using a different API key on the same account counts against the same budget. That's a common reason for a customer to say nothing changed on their side when the errors started.

I'd check internally whether their volume justifies raising the limit, and I'd ask what their sending pattern looks like, since a burst around an event needs a different answer from steady traffic.

Draft response to the customer:

> Hi there,
>
> Thanks for reaching out, and I'm happy to help. I'm Ivan, from the Resend support team.
>
> What's happening here is that you're going over the rate limit on your account, which is 2 requests per second. The requests that come back with that error aren't being accepted, which is why those users never receive the notification.
>
> There are two ways to approach it, one you can do right away and one that's more structural.
>
> The immediate fix is to put the sends into a small queue, rather than firing a request every time an event happens, so that they go out within your limit. When you do get that error, the response includes a `retry-after` header telling you how many seconds to wait before trying again. A request that was rejected was never accepted, so retrying it is safe and won't send the email twice.

The longer-term option is that if your real volume is simply above 2 per second, we can look at raising the limit on your account. If you let me know your typical and peak sends per minute, I'll take that forward.

Two other things worth knowing. The limit is shared across all the API keys on your team, so if you have another service sending on the same account, it counts against the same budget. And every response includes `ratelimit-limit`, `ratelimit-remaining` and `ratelimit-reset` headers, so your code can pace itself using those rather than relying on a fixed number. If you do automate the retries, it's worth attaching an `Idempotency-Key` header to each notification. We treat the same key as the same email for 24 hours, so a retry can't result in the same message going out twice.

Best, Ivan

How I would stop this coming back:

An optional retry helper in the official SDKs, which would honour `retry-after`, stop after a set number of attempts, and require an idempotency key. I'd make it something the customer turns on rather than the default, because retrying automatically without being asked changes how their sending behaves. Most customers hit this once and then build the same queue themselves, and a helper would save them that work.

Ticket RES-2196. "My emails are going to the spam folder at Gmail. What can I do to stop this?"

Label: Deliverability / Gmail. **Priority: 4.**

Note explaining my thinking:

The ticket tells me the mail is arriving at Gmail and being filed as spam, and nothing else. I don't know their volume, what kind of email it is, how old the domain is, or whether their authentication is set up correctly.

Spam placement has a lot of possible causes and they interact with each other, so I don't want to send a generic checklist and hope. The things that matter are authentication, how new and established the sending domain is, the content and where the links point, the quality of the list, and how recipients have engaged with previous emails. A single test send doesn't prove anything either way, and nobody can honestly promise that a message will land in the inbox.

What I need in order to make progress is one email ID from a message that went to spam, and the raw headers of that message as it was received. The authentication results in those headers tell me whether authentication is failing, which would be the first thing to fix. If SPF, DKIM and DMARC all passed for that message, that rules out a straightforward authentication failure for that

send, and I'd then investigate content, domain reputation, list quality and engagement, which are slower to work through, so it's worth establishing early which side of that line we're on.

I'd also check their domain verification and DMARC status from my side while I wait.

Draft response to the customer:

> Hi there,
>
> Thanks for reaching out. I'm Ivan, from the Resend support team, and I'll help you work through this.
>
> Spam placement is something we can diagnose, but I'd like to work from evidence rather than send you a generic list, because the right fix depends on which signal Gmail is reacting to.
>
> Could you send me two things? The first is the email ID of a message that went to spam. The second is the raw headers of that same message as you received it. In Gmail you can get those by opening the message, clicking the three dots, and choosing "Show original". The `Authentication-Results` line in there tells us whether your authentication is passing, which is the first thing we need to establish.
>
> In the meantime, the factors that most often matter are these. Whether your domain's authentication is passing and aligned. Whether the links inside the email point to your own sending domain rather than to a third-party or shortened URL, because a mismatch there can trigger spam filters. Whether the domain is new, since a new domain sending a lot of email straight away is a common trigger. And for bulk email to Gmail, whether you have one-click unsubscribe set up and how your spam complaint rate looks, which you can see in Google Postmaster Tools.
>
> I want to be straight with you about one thing: nobody can guarantee that an email lands in the inbox, and I'm not going to tell you otherwise. What we can do is remove every negative signal that's within your control, working from what your headers actually show, and that's where I'd like to start.
>
> Best, Ivan

How I would stop this coming back:

Resend already has Deliverability Insights, which covers authentication, link domains and DMARC on a per-message basis, so the feature exists. What seems to be missing is the route to it. Linking Insights directly from the places where customers ask about spam, including from the AI agent's answers to this kind of question, would let the customer start from their own evidence instead of writing to us.

Ticket RES-1927. "I'm not sure how to add the TXT record at Vercel. Can you tell me how?"

Label: Usability / Domains. **Priority: 5.**

Note explaining my thinking:

The customer is partway through verifying a domain and has got stuck at the DNS step, with their DNS hosted at Vercel. This is a quick one to answer.

The mistake I'd expect here is pasting the full hostname into Vercel's Name field. Vercel adds the domain automatically, so pasting the whole thing creates a record with the domain repeated twice and verification fails, and the saved record looks close enough to right that the customer has no reason to suspect it.

Before answering I'd confirm that Vercel is actually where their DNS is hosted, since customers sometimes buy a domain in one place and host the DNS somewhere else.

Draft response to the customer:

> Hi there,
>
> Thanks for reaching out, and happy to walk you through it. I'm Ivan, from the Resend support team.
>
> There are two ways to do this, and the first is much faster.
>
> The automatic way: on your domain's page in Resend there's an "Auto Configure" option for Vercel that sets the records up for you. If your DNS is hosted at Vercel, that's the quickest route.
>
> The manual way: in Vercel, go to your team's Domains section rather than the project, select your domain, and open DNS Records. Add a record with the type set to TXT. In the Name field, enter only the part that comes before your domain. So if Resend shows you `resend._domainkey.send.yourdomain.com`, the Name you enter is `resend._domainkey.send`. Vercel adds your domain on the end automatically, and pasting the full hostname is a common reason verification fails, because it ends up creating the record with your domain in it twice. Then paste the value exactly as Resend shows it, save, and click Verify back in Resend. It usually confirms within a few minutes.
>
> If it hasn't verified after half an hour or so, send me a screenshot of how the record looks saved in Vercel and I'll take a look at what's different.
>
> Best, Ivan

How I would stop this coming back:

The domain page could detect where the customer's DNS is hosted and show the exact values for that provider, with the warning about the name being added automatically shown inline. This same question comes up for every DNS host, so answering it in the interface would remove a whole category of ticket.

Ticket RES-5842. "I need to be able to receive emails from Resend. How do I do that?"

Label: Usability / Receiving. **Priority: 6.**

Note explaining my thinking:

The wording is ambiguous. The most likely reading is that they want to receive incoming email at their own domain through Resend, but it could also mean they want to receive notifications from us. I'll answer the likely reading and leave the door open for the other one.

The thing they most need to understand before building anything is that receiving in Resend isn't a hosted mailbox, even though it can show them messages and reply to them. Every account already has an address at `<id>.resend.app`, and messages sent there show up in the Receiving tab in the dashboard without any setup at all, which makes it very easy to try. For an application, the `email.received` webhook carries the email ID and the message metadata only. It doesn't carry the body, the headers or the attachments. The application uses that ID to fetch the body and headers from the Receiving API and the files from the Attachments API, and those attachment download links are only valid for an hour, which matters for how they build it.

If they want to receive at their own domain, that's an MX record. If their domain already handles normal email, I'd suggest pointing the MX at a subdomain instead, so their existing mail isn't affected.

Draft response to the customer:

> Hi there,
>
> Thanks for reaching out. I'm Ivan, from the Resend support team, and yes, this is something we can do.
>
> The quickest way to try it doesn't need any setup at all. Your account already has a receiving address: open Emails, go to the Receiving tab, and copy your `<id>.resend.app` address. Anything sent to any username at that address will appear right there in the dashboard.
>
> If you want your application to react to incoming mail, you can subscribe a webhook to the `email.received` event. One thing worth knowing before you build it: that event carries the message metadata only, so the email ID, sender, recipients, subject and the names of any attachments. It doesn't carry the body, the headers or the attachment contents. Your endpoint

uses the email ID to fetch the body and headers from the Receiving API, and any files from the Attachments API. Those attachment download URLs are valid for one hour, so if you need to keep a file, download it when the webhook arrives and store it your side rather than saving the link and using it later.

To receive at your own domain instead, you add the MX record shown in the receiving section. If your domain is already handling normal email, I'd suggest pointing the MX at a subdomain such as `inbound.yourdomain.com` rather than at the root, so that your existing mailboxes aren't affected.

One last thing, in case it's relevant. Resend does show received messages in the dashboard and it does support replying in the same thread, so this isn't one-way. What it isn't is a hosted mailbox with folders and a compose window for someone to work in day to day, so if that's what you need you'd want to pair it with a mailbox provider. If you tell me a bit about what you're building, I can confirm which of these fits.

Best, Ivan

How I would stop this coming back:

"Receive emails" means two different things, so this question will keep arriving. A short entry in the receiving documentation, saying plainly what Resend is and isn't in terms of inboxes, would let both the docs and the AI agent answer it without a person being involved.

Ticket RES-2984. "How do i create an email?"

Label: Usability / Sending. **Priority: 7.**

Note explaining my thinking:

There's almost nothing here to work with. "Create an email" could mean sending one from their code, designing how one looks, sending one to a list of subscribers, or setting up an address to send from. The answer is completely different in each case, and guessing wrong wastes their time and mine.

The last of those has a nice answer, which is that Resend doesn't need individual sender addresses to be registered. Once the domain is verified they can send from any address on it.

I'll ask which one they mean, but I won't send a bare question, because that just adds a round trip. I'll give them the fastest route for each possibility, so they can get moving on their own regardless of which one they meant.

Draft response to the customer:

Hi there,

Thanks for reaching out, and I'm happy to help. I'm Ivan, from the Resend support team.

I want to point you at the right thing, and "create an email" can mean a few different things here, so let me cover the most likely ones and you can tell me which is closest.

If you want to send an email from your code, our quickstart takes you from creating an API key to a sent email in a few minutes, in whichever language you're working in.

If you want to design how the email looks, you can use templates, or React Email if you'd rather build your emails in code.

If you want to send one message out to a list of subscribers, that's Broadcasts, which you can create directly in the dashboard without writing any code.

And if what you need is an address to send from, such as support@yourdomain.com, there's nothing to set up for each address. Once your domain is verified, you can send from any address on it.

Let me know which of those you're after, and a little about what you're building, and I'll give you the exact steps rather than just links.

Best, Ivan

How I would stop this coming back:

This is the clearest case for the AI agent handling it without a person. A short set of clarifying options, like the ones above, resolves it. If questions this general are reaching a human, then either the agent's first response or the entry point in the documentation needs the same disambiguation that this reply does.

Escalation message to the engineering team, for RES-7921

To: engineering on-call **From:** support (Ivan) **Severity:** High. I'd suggest treating this as an incident until we can confirm the scope. **Subject:** Around four hours of failed sending last night on account [ACCT-ID], thousands of login emails affected. Scope not yet known.

Summary: the customer reports that their sending stopped for roughly four hours last night. Their estimate of the window is approximately 22:00 to 02:00 UTC, which I haven't been able to confirm yet, and anything I haven't confirmed is marked below. They're reporting thousands of missing emails. The emails are login magic links, so each missing message meant somebody couldn't get into their account. Sending appears to have recovered.

The bug, based on what we have so far:

[ASSUMED FOR EXERCISE] During the affected window, a sample of `POST /emails` requests returned a 200 along with email IDs, and those messages recorded an `email.sent` event, which only tells us the API request was accepted. They then recorded nothing further. No `email.delivered`, no `email.bounced`, no `email.failed` and no `email.suppressed`, for around four hours. The same request worked before the window and after it.

- Expected: a message we accept moves through the pipeline and reaches some final delivery outcome, whether that's delivered, bounced, failed or suppressed.
- Actual: the messages were accepted and stopped there, with no outcome recorded and no error shown to the customer.
- The bug: accepted messages that never reached any outcome at all. It's worth being clear about the distinction, because `email.sent` is easy to misread as success. It means we took the request, not that we delivered anything. An email that bounces or gets delayed doesn't by itself establish a bug on our side, because the receiving server can cause both. Something we accepted and then never resolved either way can't be explained from the receiving end, which is what makes it ours.

What I've been able to check so far:

- The account's sending graph shows [ASSUMED: a drop to zero within the reported window, with normal volume either side]
- Status page for that window: [PENDING]. The customer said "last night" without giving a date, so I haven't been able to confirm this yet.
- Sample email IDs from the customer: [PENDING]

How to reproduce and confirm: pull the API request logs for [ACCT-ID] across that window and split them by response status. For a sample of the accepted IDs, check whether the delivery events are missing. Then run the same query without filtering by account, in order to establish whether this affected only this customer or was wider.

Why I think this is urgent even though sending has recovered:

1. We don't know the cause, which means we don't know whether it will happen again tonight at the same time.
2. If any other accounts show the same window, this is an incident that wasn't reported, and it needs a status page entry and a postmortem rather than a support reply.
3. The customer is asking what happened and deserves a real answer. I'm holding them with an investigation update and have committed to coming back to them hourly.
4. They may not be able to safely resend the failed batch, since magic links normally carry an expiry and we don't yet know theirs, so "just send them again" may not be available as a workaround. I've asked them to hold off until we've confirmed it.

What I'm asking for: confirmation of the scope within the hour if that's possible, the root cause once it's known, and a decision on whether this needs a status page entry if it turns out to be wider than this one account. I'll handle all of the communication with the customer.

Some details above are marked [ASSUMED] or [PENDING], since the ticket as given doesn't include logs, an account ID or timestamps.